Maturity Wall Simulator :אפיון · Case 02

פרק 2 (אפיון) בתבנית תיק הפרויקט של מגשימים: פיצ'רים, טכנולוגיות, תרשים זרימה ומבנה בסיס נתונים. המסמך כתוב כך שמפתח אחר יוכל לבנות את המערכת ממנו. כל פיצ'ר מפורט ברמת "קוד במילים": מה הוא עושה, איך הוא עובד שלב אחר שלב, אילו פונקציות ומבני נתונים, אילו כלים מהתעשייה משלימים אותו, ומה הקלט והפלט שלו.

סקירת המערכת

מה המערכת עושה, במשפט אחד

Maturity Wall Simulator טוען את רשימת האג"ח הממשלתיות של מדינה ממקור רשמי, אג"ח אחר אג"ח, ומחשב קדימה איך כל אחת מהן תוחלף, באיזו ריבית, ומה זה יעשה לתשלומי הריבית ולחוב, תחת כל תרחיש תשואות שהמשתמש בוחר.

מה השתנה מאז ה-POC ומודל האיום

- ה-POC הוכיח שחישוב אג"ח אחר אג"ח נותן תשובה שונה מהקירובים הפשוטים. שאלה פתוחה שנשארה: האם גם מול קירוב מקצועי בסגנון IMF? לפני האפיון בדקנו: קירוב בסגנון IMF שמשתמש בפרופיל הפקיעות האמיתי עדיין טעה בעד 0.53 נקודות אחוז מהתמ"ג בתרחיש "התשואות נשארות", ובעד 0.8 בתרחיש "עלייה". הסיבה העיקרית: אג"ח קצרות שהונפקו אחרי היום פוקעות שוב ושוב, והקירוב לא רואה את זה. פיצ'ר 7 הופך את הבדיקה הזו לחלק מהמערכת.
- **מודל האיום קבע שהאיום המרכזי הוא אמינות:** מספר שמצוטט חייב להוכיח מאיפה הגיע. פיצ'רים 2, 3 ו-9 בנויים סביב זה.

רשימת הפיצ'רים

- פיצ'ר 1, איסוף ממקורות רשמיים (Source Adapters)
- פיצ'ר 2, נרמול ובדיקת שלמות (Normalization & Reconciliation)
- פיצ'ר 3, מאגר גרסאות ושרשרת מקור (Versioned Store & Provenance)
- פיצ'ר 4, בניית עקום תשואות ותרחישים (Curve & Scenario Builder)
- פיצ'ר 5, מנוע המחזור (Rollover Engine)
- פיצ'ר 6, מכונת דינמיקת החוב ו-r מול g (Debt Dynamics Engine)
- פיצ'ר 7, השוואת שיטות (Method Comparison)
- פיצ'ר 8, קיר הפקיעות האינטראקטיבי (Maturity Wall View)
- פיצ'ר 9, קישור משחזר וחותמת תוצאה (Reproducible Permalinks & Signed Results)
- פיצ'ר 10, רענון מתוזמן וזיהוי שינויים (Scheduled Refresh & Change Detection)

פיצ'ר 1: איסוף ממקורות רשמיים (Source Adapters)

מה הפיצ'ר עושה?

מוריד את רשימת האג"ח הסחירות של כל מדינה ממשרד החוב שלה, ואת עקום התשואות הנוכחי, וממיר כל מקור לפורמט ביניים אחיד. כל מדינה מפרסמת אחרת: ארה"ב דרך API ציבורי, צרפת ואיטליה בקובצי Excel חודשיים. הפיצ'ר מסתיר את ההבדלים האלה משאר המערכת.

איך הפיצ'ר עובד?

ממשק אחיד לכל מקור

כל מקור ממומש כמחלקה שיורשת מ-SourceAdapter, מחלקה אבסטרקטית עם שלוש מתודות: fetch() שמורידה את הקובץ הגולמי ושומרת אותו כמו שהוא, parse() שממירה אותו לרשימת RawBond, ו-declared_total() שמחזירה את סך החוב הסחיר שהמדינה עצמה מדווחת (לצורך בדיקת השלמות בפיצ'ר 2). המחלקות נרשמות במילון ADAPTERS לפי קוד מדינה (ISO 3166, למשל "IT", "FR", "US"), כך שהוספת מדינה היא הוספת מחלקה אחת בלי לגעת בשאר הקוד.

ארה"ב: Fiscal Data API

משרד האוצר האמריקאי מפרסם את טבלה 3 של הדוח החודשי על החוב הציבורי (Monthly Statement of the Public Debt, Detail of Marketable Treasury Securities Outstanding) דרך ה-API הציבורי שלו, ב-endpoint בשם v1/debt/mspd/mspd_table_3_market. כל רשומה מתארת סדרת אג"ח אחת, עם השדות security_class1_desc (סוג: Bills, Notes, Bonds, TIPS, FRN), security_class2_desc (מזהה CUSIP), interest_rate_pct (קופון), yield_pct, issue_date, maturity_date, outstanding_amt (במיליוני דולרים) ו-inflation_adj_amt (תוספת הצמדה ל-TIPS).

המתאם שולח בקשת GET עם הפרמטרים <filter=record_date:eq:<YYYY-MM-DD (סוף החודש האחרון שפורסם) ו-page[size]=10000, ומדפדף לפי השדה links.next בתשובה עד שאין עוד עמודים. הבקשות נשלחות דרך httpx עם timeout של 30 שניות, עד 3 ניסיונות חוזרים עם המתנה אקספוננציאלית (backoff של 2, 4 ו-8 שניות), וכותרת User-Agent מזהה.

צרפת ואיטליה: קובצי Excel חודשיים

ה-Agence France Trésor מפרסמת כל חודש קובץ של החוב הסחיר (OAT ארוכות, BTF קצרות, ו-OATi/OAT€i צמודות). משרד האוצר האיטלקי מפרסם קובץ דומה ל-BTP ו-BOT. המתאם מוריד את הקובץ, שומר אותו כמו שהוא, וקורא אותו עם openpyxl. מכיוון שמבנה הגיליון יכול להשתנות, המתאם לא מסתמך על מספרי עמודות: הוא מחפש את שורת הכותרות לפי מילות מפתח ("Échéance", "Encours", "Coupon"), ובונה מיפוי עמודות דינמי. אם לא נמצאו כל הכותרות הנדרשות, המתאם נכשל במפורש ולא מנחש.

עקום התשואות

לאירופה: הבנק המרכזי האירופי מפרסם במאגר YC את פרמטרי עקום Svensson היומיים של אג"ח ממשלתיות בגוש האירו (סדרות כמו YC.B.U2.EUR.4F.G_N_A.SV_C_YM.BETA0 עד TAU2 לעקום של מדינות AAA, ו-G_N_C לכל המדינות). המתאם מוריד את ששת הפרמטרים של היום האחרון. לפער הספציפי של כל מדינה מעל העקום האירופי (המרווח), המערכת משתמשת בתשואות של האג"ח של אותה מדינה עצמה, שכבר קיימות ברשימה. לארה"ב: עקום ה-Par Yield היומי של משרד האוצר.

שמירת המקור הגולמי

כל קובץ גולמי נשמר כמו שהוא, לפני כל עיבוד, עם טביעת אצבע SHA-256 של התוכן ותאריך ההורדה. זה הבסיס לשרשרת המקור בפיצ'ר 3: תמיד אפשר לחזור לקובץ המקורי שממנו נגזר מספר.

טכנולוגיות ותשתיות תעשייה משלימות

- **httpx עם tenacity:** httpx היא ספריית HTTP מודרנית לפייתון שתומכת בעבודה סינכרונית וא-סינכרונית, ב-HTTP/2, ובניהול timeouts מדויק. tenacity היא ספרייה ייעודית לניסיונות חוזרים עם מדיניות backoff. התרומה: מקור רשמי שלא זמין לרגע לא מפיל את כל הרענון, וניסיון חוזר לא מציף את השרת שלהם.
- **Pydantic:** ספריית אימות נתונים שבה כל מבנה מוגדר כמחלקה עם טיפוסים. כל RawBond שהמתאם מחזיר עובר אימות טיפוסים מיידי (תאריך הוא תאריך, סכום הוא מספר חיובי). התרומה: שגיאת פרסור נתפסת ליד המקור שלה, ולא שלושה שלבים אחר כך כמספר מוזר בגרף.

- **Great Expectations (כהשראה):** כלי מוביל בתעשייה ל"בדיקות איכות נתונים" (data contracts). אנחנו מאמצים את הרעיון: לכל מקור יש "חוזה" של ציפיות (טווח קופונים סביר, תאריך פקיעה אחרי תאריך הנפקה), שנבדק בכל טעינה.

מבנה נתונים

```
class RawBond(BaseModel):
    source_id: str          # "US_MSPD_T3", "FR_AFT_MONTHLY"
    instrument_id: str      # CUSIP / ISIN
    kind: Literal["bill", "fixed", "linker", "floater"]
    currency: str           # ISO 4217
    coupon_pct: float | None
    issue_date: date
    maturity_date: date
    outstanding: Decimal    # in currency units, not millions
    index_ratio: float | None   # linkers only

class FetchRecord:
    source_id: str
    url: str
    fetched_at: datetime
    raw_sha256: str
    raw_path: str
```

קלט ופלט

קלט: קוד מדינה ותאריך דוח. פלט: רשימת RawBond, סך החוב המוצהר, עקום תשואות גולמי, ורשומת FetchRecord לכל קובץ שהורד.

ספריות ופונקציות מרכזיות

שימוש	ספרייה / API	פעולה
client.get() עם params ודפדוף	httpx	הורדה מ-API
retry(wait=wait_exponential, @ stop=stop_after_attempt(3))	tenacity	ניסיונות חוזרים
load_workbook(read_only=True), iter_rows()	openpyxl	קריאת Excel
BaseModel, field_validator	Pydantic	אימות טיפוסים
sha256(raw_bytes).hexdigest()	hashlib	טביעת אצבע

פיצ'ר 2: נרמול ובדיקת שלמות (Normalization & Reconciliation)

מה הפיצ'ר עושה?

ממיר את ה-RawBond של כל מדינה לאובייקט Bond אחד, אחיד ונקי, ומוודא שהרשימה שלמה ונכונה לפני שמשהו נבנה עליה. זה קו ההגנה מול האיום "שיבוש בטעינה" ממודל האיום: מתאם שקרא עמודה לא נכונה, או קובץ שהשתנה בשקט.

איך הפיצ'ר עובד?

נרמול

כל סכום מומר מהיחידה של המקור (מיליונים, מיליארדים) ליחידות מטבע מלאות, ונשמר כ-Decimal ולא כ-float, כדי למנוע שגיאות עיגול כשמחברים אלפי אג"ח. תאריכים מנורמלים ל-date. לאג"ח צמודות מדד (linker) נשמר גם הקרן המקורית וגם הקרן המותאמת (outstanding כפול index_ratio), כי תשלום הריבית מחושב על המותאמת. מזהים כפולים (אותו ISIN בשתי שורות, למשל בגלל הנפקה חוזרת של אותה סדרה) מאוחדים לסדרה אחת עם סכום מצטבר.

בדיקת שלמות מול החוב המוצהר

אחרי הנרמול, הפיצ'ר מחבר את כל ה-outstanding ומשווה ל-declared_total() של אותה מדינה. סף הסבולת: 0.5%. מעל הסף, הטעינה מסומנת REJECTED ולא נכנסת למאגר. מתחת לסף, הפער נשמר כשדה reconciliation_gap ומוצג בממשק ליד כל תוצאה ("הרשימה מכסה 99.7% מהחוב המדווח").

בדיקות סבירות (data contract)

לכל אג"ח: maturity_date אחרי issue_date; קופון בין 0 ל-15 אחוז (ריק רק ל-bill); outstanding חיובי; מטבע תואם למדינה. לכל הטעינה: מספר האג"ח לא השתנה ביותר מ-10% מהטעינה הקודמת; הקופון הממוצע המשוקלל לא זז ביותר מ-0.3 נקודות אחוז בחודש. כל הפרה נרשמת כ-ValidationIssue עם חומרה (WARN או BLOCK). הפרה בחומרת BLOCK עוצרת את הטעינה.

השוואה לטעינה הקודמת

הפיצ'ר מחשב diff בין הרשימה החדשה לקודמת לפי instrument_id: אג"ח שנוספו (הנפקות חדשות), אג"ח שנעלמו (פקעו או נפדו), ואג"ח שהסכום שלהן השתנה. אג"ח שנעלמה לפני תאריך הפקיעה שלה היא חריגה שמסומנת לבדיקה (ייתכן פדיון מוקדם, וייתכן שגיאת קריאה).

טכנולוגיות ותשתיות תעשייה משלימות

- **Pandera:** ספריית סכמות לטבלאות נתונים (DataFrame) שמגדירה כללים לכל עמודה ולטבלה כולה, ובודקת אותם בבת אחת. התרומה: כללי הסבירות כתובים פעם אחת כסכמה מוצהרת, ולא מפוזרים כ-if בקוד.
- **Polars:** ספריית טבלאות עמודתית, כתובה ב-Rust. ה-diff בין שתי טעינות הוא join לפי instrument_id, ו-Polars מבצעת אותו במהירות גם על עשרות אלפי שורות. נבחרה על פני pandas בגלל טיפוסים קפדניים יותר וביצועים.
- **עקרון "Reconciliation" מעולם הבנקאות:** בכל מערכת פיננסית רצינית, כל עיבוד מסתיים בהתאמה מול מקור עצמאי ("סגירת ספרים"). אנחנו מאמצים את העיקרון כפשוטו: אין תוצאה בלי התאמה מול מה שהמדינה עצמה מצהירה.

מבנה נתונים

```
@dataclass(frozen=True)
class Bond:
    instrument_id: str
    country: str
    kind: str              # bill | fixed | linker | floater
    coupon_pct: Decimal
    issue_date: date
    maturity_date: date
    face: Decimal          # outstanding, inflation-adjusted for linkers
    orig_tenor_y: int      # rounded (maturity - issue) in years

class ValidationIssue:
    rule_id: str           # "COUPON_RANGE", "COUNT_JUMP", ...
```

```
    severity: Literal["WARN", "BLOCK"]
    instrument_id: str | None
    detail: str
```

קלט ופלט

קלט: רשימת RawBond וסך מוצהר מפיצ'ר 1, והטעינה הקודמת מהמאגר. פלט: רשימת Bond מאומתת, רשימת ValidationIssue, ו-diff מול הטעינה הקודמת. או החלטת REJECTED עם reconciliation_gap הסיבות.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
חישוב מדויק	decimal	()Decimal, quantize
סכמות ובדיקות	Pandera	DataFrameSchema, Check
השוואה בין טעינות	Polars	()join(how="full"), filter

פיצ'ר 3: מאגר גרסאות ושרשרת מקור (Versioned Store & Provenance)

מה הפיצ'ר עושה?

שומר כל טעינה מאושרת כ"תמונת מצב" (snapshot) שאינה משתנה לעולם, ומחזיק לכל תמונה את כל השרשרת: מאיזה קובץ גולמי היא נוצרה, באיזו גרסת מתאם, ואילו בדיקות עברה. כך כל מספר שהמערכת מוציאה ניתן לעקוב עד לקובץ הרשמי שממנו הגיע. זה המענה לאיום "הכחשת מקור".

איך הפיצ'ר עובד?

תמונות מצב בלתי משתנות

כל טעינה מאושרת מקבלת snapshot_id, שהוא טביעת אצבע SHA-256 של רשימת ה-Bond המנורמלת. כדי שהטביעה תהיה דטרמיניסטית, הרשימה ממוינת לפי instrument_id ומסודרת ל-JSON קנוני (מפתחות ממוינים, בלי רווחים, מספרים כמחרוזות Decimal) לפני החישוב. אותה רשימה תיתן תמיד אותו מזהה, ושינוי של אג"ח אחת ישנה אותו. רשומות לעולם לא נמחקות ולא מתעדכנות: טעינה חדשה היא snapshot חדש.

שרשרת המקור

לכל snapshot נשמרת רשומת Provenance: מזהי FetchRecord של הקבצים הגולמיים (עם ה-SHA-256 שלהם), גרסת קוד המתאם (git commit), רשימת ValidationIssue, ו-reconciliation_gap. המבנה בהשראת תקן W3C PROV: ישות (snapshot) נגזרה מישות (קובץ גולמי) על ידי פעילות (טעינה) שבוצעה על ידי סוכן (מתאם בגרסה מסוימת).

אחסון

המאגר הוא DuckDB, בסיס נתונים אנליטי שרץ בתוך התהליך ושומר הכול בקובץ אחד. הקבצים הגולמיים נשמרים בתיקייה לפי raw_sha256 (content-addressed storage), כך שקובץ זהה שהורד פעמיים נשמר פעם אחת.

טכנולוגיות ותשתיות תעשייה משלימות

- **DuckDB:** "SQLite של האנליטיקה". רץ בתוך התהליך בלי שרת, עמודתי, ומהיר מאוד בשאילתות אגרגציה על טבלאות גדולות. התרומה: כל ההיסטוריה של כל המדינות נשאלת ב-SQL רגיל, וניתנת לייצוא כקובץ Parquet אחד לכל snapshot.
- **W3C PROV:** תקן לתיאור מקור (provenance) של נתונים, שמשמש מאגרי מחקר וממשלות. אנחנו לא מממשים את כל התקן, רק את מודל שלושת המושגים שלו (Entity, Activity, Agent). התרומה: מבנה מוכר וברור במקום המצאה.
- **Content-Addressed Storage:** העיקרון שעליו בנוי Git: קובץ מזוהה לפי טביעת האצבע של התוכן שלו. אי אפשר לשנות קובץ בלי שהמזהה שלו ישתנה, ולכן "שינוי שקט" בלתי אפשרי.

מבנה נתונים

```
Snapshot {
  snapshot_id: str          # sha256 of canonical bond list
  country: str
  as_of: date               # report date
  created_at: datetime
  bond_count: int
  total_face: Decimal
  reconciliation_gap_pct: float
}

Provenance {
  snapshot_id: str
  fetch_ids: list[str]      # -> FetchRecord
  adapter_version: str      # git commit sha
  issues: list[ValidationIssue]
}
```

קלט ופלט

קלט: רשימת Bond מאומתת ונתוני הבדיקה מפיצ'ר 2. פלט: snapshot_id קבוע, ושורות במאגר שלעולם לא משתנות.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
מאגר אנליטי	DuckDB	duckdb.connect(), execute(), Parquet export
קנוני JSON	json	dumps(sort_keys=True, separators=(",", ":"))
טביעת אצבע	hashlib	sha256

פיצ'ר 4: בניית עקום תשואות ותרחישים (Curve & Scenario Builder)

מה הפיצ'ר עושה?

מנוע המחזור צריך לדעת, לכל שנה עתידית ולכל אורך אג"ח, באיזו תשואה תונפק אג"ח חדשה. הפיצ'ר בונה עקום תשואות רציף מהנתונים של היום, ומאפשר להגדיר תרחישים: איך העקום הזה זז לאורך עשר השנים הבאות.

איך הפיצ'ר עובד?

עקום Nelson-Siegel-Svensson

עקום תשואות הוא פונקציה שמחזירה תשואה לכל אורך זמן (שנה, 5 שנים, 30 שנה). השיטה המקובלת אצל בנקים מרכזיים, כולל ה-ECB, היא מודל Nelson-Siegel-Svensson: שישה פרמטרים (ארבעה מקדמי בטא ושני מקדמי טאו) שמתארים את הרמה, השיפוע והעיקול של העקום. לאירופה הפרמטרים מגיעים מוכנים מה-ECB. למרווח של כל מדינה מעל העקום האירופי, הפיצ'ר מתאים עקום משלו לתשואות האג"ח של אותה מדינה (yield_pct או תשואה מחושבת ממחיר), באמצעות scipy.optimize.least_squares שממזער את סכום ריבועי הטעויות בין התשואה של העקום לתשואה בפועל של כל אג"ח. התוצאה: פונקציה curve(tenor) שמחזירה תשואה לכל אורך.

תרחישים

תרחיש הוא מסלול של שינויים בעקום לאורך זמן, שנה אחר שנה. ארבעה סוגים מובנים:

- **Parallel shift:** כל העקום זז באותו שיעור (למשל עלייה של נקודת אחוז).
- **Steepener / Flattener:** התשואות הארוכות זזות יותר או פחות מהקצרות.
- **Path:** מסלול מותאם, למשל עלייה הדרגתית של רבע נקודה בשנה.
- **Forward:** העקום העתידי שהשוק עצמו מגלם היום (forward rates), כלומר "מה השוק מצפה".

לכל תרחיש נלווים גם הנחות מאקרו: צמיחה נומינלית ומאזן ראשוני לכל שנה. ברירות המחדל לקוחות מתחזיות רשמיות (IMF World Economic Outlook, ותחזיות הנציבות האירופית), עם קישור למקור.

תמהיל ההנפקה

באיזה אורך תונפק אג"ח שמחליפה אג"ח שפקעה? משרד החוב מחליט. ברירת המחדל: התמהיל של שלוש השנים האחרונות, כפי שהוא נגזר מהאג"ח שהונפקו בתקופה הזו ברשימה עצמה. המשתמש יכול לשנות אותו (למשל "צרפת עוברת להנפיק יותר לטווח קצר"). התמהיל הוא פרמטר גלוי ולא הנחה חבויה, כפי שקבע ה-POC.

טכנולוגיות ותשתיות תעשייה משלימות

- **SciPy least_squares) עם שיטת Trust Region Reflective):** פותרת בעיות ריבועים פחותים לא-ליניאריות עם גבולות לפרמטרים (למשל tau חיובי). התרומה: התאמת עקום יציבה ומהירה, שלא בורחת לפרמטרים חסרי משמעות.
- **QuantLib (כחלופה וכבדיקה):** הספרייה הפתוחה הוותיקה והנפוצה בעולם לתמחור נגזרים ואג"ח, שמשמשת בנקים ומוסדות. כוללת בניית עקומים (bootstrapping) ומודלי NSS. התרומה: בדיקה צולבת. העקום שבנינו מושווה לעקום ש-QuantLib בונה מאותם נתונים, והפער חייב להיות זניח.
- **מודל Svensson של ה-ECB:** שימוש באותה שיטה שבה הבנק המרכזי עצמו מפרסם את העקום. התרומה: קורא מקצועי יכול להשוות את העקום שלנו לפרסום הרשמי.

מבנה נתונים

```
NSSParams { beta0, beta1, beta2, beta3: float; tau1, tau2: float }

Curve {
  curve_id: str
  base: Literal["ECB_AAA", "ECB_ALL", "UST_PAR", "FITTED"]
  as_of: date
  params: NSSParams
  spread_params: NSSParams | None   # country spread over base
  fit_rmse_bp: float
}
```

```
Scenario {
  scenario_id: str           # sha256 of canonical definition
  name: str
  curve_id: str
  shocks: list[YearShock]    # per year: level_bp, slope_bp
  growth_nominal_pct: list[float]
  primary_balance_pct: list[float]
  issuance_mix: dict[int, float]   # tenor -> share, sums to 1
  source_notes: list[str]
}
```

קלט ופלט

קלט: פרמטרי עקום מה-ECB או משרד האוצר, תשואות האג"ח מהרשימה, ובחירות המשתמש. פלט: Curve מותאם, Scenario עם מזהה קבוע, ופונקציה yield_at(year, tenor).

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
optimize.least_squares(method="trf", bounds=...)	SciPy	התאמת עקום
FittedBondDiscountCurve, SvenssonFitting	QuantLib	בדיקה צולבת
הערכת העקום על כל האורכים בבת אחת	NumPy	חישוב וקטורי

פיצ'ר 5: מנוע המחזור (Rollover Engine)

מה הפיצ'ר עושה?

זו הליבה הטכנולוגית. המנוע מקבל snapshot ותרחיש, ומריץ את העתיד חודש אחר חודש: כל אג"ח שפוקעת מוחלפת באג"ח חדשה בתשואת התרחיש, כל גירעון ממומן בהנפקה חדשה, וכל תשלום קופון נרשם. הפלט הוא המסלול המלא של החוב ותשלומי הריבית, ברמת אג"ח בודדת.

איך הפיצ'ר עובד?

צעד זמן חודשי

ה-POC עבד בצעדים שנתיים. במערכת המלאה הצעד חודשי, כי קירות פקיעה מתרכזים בחודשים מסוימים (למשל אג"ח אמריקאיות שפוקעות ב-15 של פברואר, מאי, אוגוסט ונובמבר), ואג"ח קצרות (bills) לחודש, שלושה ושישה חודשים) מתגלגלות כמה פעמים בשנה. עשר שנים הן 120 צעדים.

מבנה הנתונים בזיכרון

במקום רשימה של אובייקטים, התיק מוחזק כמערכי NumPy מקבילים (Structure of Arrays): face, coupon, maturity_month, issue_month, kind, tenor. כך כל פעולה על כל התיק היא פעולה וקטורית אחת. כדי למצוא מהר מה פוקע בכל חודש, נבנה מילון maturing_index מחודש לרשימת אינדקסים, שמתעדכן כשמונפקת אג"ח חדשה. זה הופך את חיפוש הפקיעות מ-O(n) לכל צעד ל-O(1).

מה קורה בכל צעד

- **ריבית:** חישוב הקופונים של החודש לכל האג"ח הפעילות. אג"ח קבועה משלמת קופון לפי לוח התשלומים שלה (חצי-שנתי בארה"ב, שנתי באירופה). bill לא משלם קופון, והעלות שלו היא ההנחה בהנפקה (

discount), שנרשמת כריבית בחודש הפקיעה. אג"ח צמודה משלמת קופון על קרן מותאמת, והקרן עצמה גדלה עם האינפלציה של התרחיש. floater משלם לפי הריבית הקצרה של התרחיש באותו חודש.

- **פקיעות:** כל האג"ח שבאינדקס של החודש יוצאות מהתיק.
- **צורך מימון:** סכום הפקיעות, פלוס הגירעון הכולל של החודש (הגירעון הראשוני של התרחיש חלקי 12, פלוס הריבית ששולמה).
- **הנפקה:** הצורך מחולק לפי תמהיל ההנפקה. לכל אורך נוצרת אג"ח חדשה עם קופון שווה ל-yield_at(month, tenor) מהעקום של התרחיש, מעוגל ל-0.125 כמקובל בהנפקות אמיתיות.
- **רישום:** לכל צעד נשמרים סך החוב, הריבית ששולמה, הריבית האפקטיבית (ריבית שנתית חלקי חוב), והפקיעות לפי אורך מקורי.

בדיקת זהות (invariant)

בכל צעד המנוע בודק את זהות המלאי-תזרים: החוב בסוף החודש שווה לחוב בתחילתו, פלוס הגירעון הכולל, פחות שום דבר אחר. סטייה של יותר מ-1 יחידת מטבע זורקת חריגה. זה מבטיח שאין "דליפות" בגלגול, כפי שנבדק כבר ב-POC.

ביצועים

עשרות אלפי אג"ח כפול 120 צעדים הם פעולות וקטוריות פשוטות. היעד: הרצה של תרחיש אחד למדינה אחת בפחות מ-200 מילישניות, כך שהמחוון בממשק יגיב בזמן אמת. כדי שזה ירוץ גם בדפדפן, המנוע כתוב בפייתון עם NumPy בלבד ונטען ב-Pyodide.

טכנולוגיות ותשתיות תעשייה משלימות

- **NumPy (Structure of Arrays):** הדפוס שבו מנועי משחקים ומערכות סימולציה מחזיקים מיליוני ישויות. התרומה: עדכון כל התיק בפעולה וקטורית אחת במקום לולאה על אובייקטים.
- **Numba (אופציונלי בצד השרת):** מהדר JIT שמתרגם פונקציות פייתון לקוד מכונה. התרומה: אם הרצות Monte Carlo בצד השרת (פיצ'ר 6) ידרשו אלפי ריצות, לולאת הצעדים תהודר ותרוץ פי עשרות מהר יותר. בדפדפן לא זמין, ולכן נשאר אופציונלי.
- **Pyodide:** הפצה של פייתון שמהודרת ל-WebAssembly ורצה בדפדפן, כולל NumPy. התרומה: אותו קוד מנוע רץ גם בשרת וגם אצל המשתמש. זה גם המענה לאיום "הפלת השירות": החישוב לא תלוי בשרת.

מבנה נתונים

```
class Portfolio:            # Structure of Arrays
    face: np.ndarray        # float64
    coupon: np.ndarray      # float64, % per year
    kind: np.ndarray        # int8: 0 bill, 1 fixed, 2 linker, 3 floater
    issue_m: np.ndarray     # int32, months since epoch
    maturity_m: np.ndarray  # int32
    tenor_m: np.ndarray     # int32, original tenor in months
    maturing_index: dict[int, list[int]]

StepResult {
  month: int
  debt: float
  interest_paid: float
  effective_rate_pct: float
  maturing_by_tenor: dict[int, float]
  issued_by_tenor: dict[int, float]
}
```

קלט ופלט

קלט: snapshot (רשימת Bond) ו-Scenario. פלט: רשימת StepResult לכל חודש, והתיק הסופי.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
מערכי תיק	NumPy	np.where, np.add.at, masking
האצה בשרת	Numba	njit@ על לולאת הצעדים
ריצה בדפדפן	Pyodide	loadPackage("numpy"), runPythonAsync

פיצ'ר 6: מכונת דינמיקת החוב ו-r מול g (Debt Dynamics Engine)

מה הפיצ'ר עושה?

מתרגם את הפלט של מנוע המחזור לשאלות שאכפת לאנשים מהן: מה יחס החוב לתמ"ג בכל שנה, מתי הריבית האפקטיבית עוקפת את הצמיחה, ואיזה עודף ראשוני נדרש כדי לעצור את התפיחה.

איך הפיצ'ר עובד?

משוואת החוב

יחס החוב לתמ"ג בשנה t שווה ליחס בשנה הקודמת, כפול (1 ועוד הריבית האפקטיבית) חלקי (1 ועוד הצמיחה הנומינלית), פחות המאזן הראשוני, ועוד "התאמות מלאי-תזרים" (stock-flow adjustments, למשל הפרטות או הלוואות לגופים ציבוריים). הריבית האפקטיבית לא מונחת אלא מגיעה ממנוע המחזור, שנה אחר שנה. זה ההבדל בינינו לבין קירוב מצרפי.

נקודת החצייה של r מול g

הפיצ'ר מחפש את השנה הראשונה שבה הריבית האפקטיבית עוברת את הצמיחה הנומינלית, ומסמן אותה. מעבר לנקודה הזו, כל שנה בלי עודף ראשוני מגדילה את יחס החוב, גם בלי גירעון "חדש".

העודף הראשוני הנדרש

לכל שנה: העודף הראשוני שהיה מייצב את יחס החוב באותה שנה שווה ל-(r פחות g) חלקי (1 ועוד g), כפול יחס החוב. ה-POC הראה שהנוסחה נותנת 1.7% לתיק המדומה, מול 1.8% שחישב The Economist לצרפת. הפיצ'ר גם מחשב את המסלול ההפוך: "איזה מאזן ראשוני קבוע מחזיר את החוב לרמה X בשנה Y", באמצעות חיפוש בינארי (brentq) על המאזן הראשוני, כשכל ניסיון מריץ את מנוע המחזור מחדש.

מניפת אי-ודאות (אופציונלי, בשרת)

במקום תרחיש יחיד, הרצה של 1,000 מסלולים שבהם התשואות והצמיחה נדגמות מהתפלגות משותפת שנאמדה מההיסטוריה (vector autoregression פשוט). התוצאה היא "מניפה" (fan chart) של יחס החוב, כמו שמפרסמים IMF והנציבות האירופית: אחוזונים 10, 50 ו-90.

טכנולוגיות ותשתיות תעשייה משלימות

- **SciPy brentq:** אלגוריתם מציאת שורש שמשלב חציה, סקנט ואינטרפולציה. התרומה: מציאת המאזן הראשוני הנדרש בכמה עשרות הרצות, במקום חיפוש עיוור.
- **statsmodels (VAR):** ספרייה סטטיסטית מוכרת שבה מודל VAR הוא כלי סטנדרטי לאמידת התנהגות משותפת של כמה סדרות. התרומה: מסלולי תשואה וצמיחה שמתואמים זה עם זה, כמו במציאות.
- **Sobol sequences (scipy.stats.qmc):** דגימה "קוואזי-אקראית" שמכסה את מרחב האפשרויות באופן אחיד יותר מדגימה אקראית רגילה. התרומה: מניפה יציבה עם פחות ריצות.

מבנה נתונים

```
DebtPath {
  run_id: str
  years: list[int]
  debt_gdp_pct: list[float]
  interest_gdp_pct: list[float]
  effective_rate_pct: list[float]
  growth_pct: list[float]
  r_minus_g_pp: list[float]
  crossover_year: int | None
  stabilizing_pb_pct: list[float]
}

FanChart { run_id; p10, p50, p90: list[float]; n_paths: int; seed: int }
```

קלט ופלט

קלט: StepResult ממנוע המחזור והנחות המאקרו של התרחיש. פלט: DebtPath, ואופציונלית FanChart.

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
optimize.brentq	SciPy	מציאת שורש
tsa.api.VAR	statsmodels	מודל מסלולים
stats.qmc.Sobol	SciPy	דגימה

פיצ'ר 7: השוואת שיטות (Method Comparison)

מה הפיצ'ר עושה?

מריץ את אותו תרחיש בארבע שיטות במקביל, ומראה איפה הן נפרדות ולמה. זה הפיצ'ר שעונה על השאלה הפתוחה מה-POC: האם הפירוט ברמת אג"ח באמת שווה משהו. הוא לא מניח את התשובה, הוא מודד אותה בכל הרצה מחדש.

איך הפיצ'ר עובד?

ארבע השיטות

- **ריבית קפואה:** כל החוב ממשיך לשלם את הריבית האפקטיבית של היום. כך עובדים חישובי "מעטפת" רבים.
- **קפיצה מיידית:** כל החוב עובר מיד לריבית השוק. זה התרגיל המחשבתי של The Economist.
- **קירוב בסגנון IMF:** ריבית אפקטיבית שמתעדכנת כל שנה לפי חלק החוב שמתחדש, כשהחלק נגזר מפרופיל הפקיעות של היום. גרסה מתקדמת מפרידה חוב קצר מחוב ארוך, כמו בתבנית ה-DSA של IMF.
- **אג"ח אחר אג"ח:** מנוע המחזור המלא (פיצ'ר 5).

מדידת הפער

לכל שנה: הפער בתשלומי הריבית (באחוזי תמ"ג) בין כל שיטה לבין המנוע המלא, והפער ביחס החוב. הפיצ'ר מסמן את השנה עם הפער הגדול ביותר, ומפרק אותו למקורות: כמה מהפער נובע מגלגול של אג"ח קצרות חדשות (שהקירוב לא רואה), כמה מאג"ח צמודות, וכמה מתזמון חודשי.

מה בדקנו כבר

לפני האפיון הרצנו קירוב בסגנון IMF (הגרסה הפשוטה, בלי הפרדה לקצר וארוך) על התיק המדומה של ה-POC. הפער מול המנוע המלא הגיע ל-0.53 נקודות אחוז מהתמ"ג בתרחיש "התשואות נשארות", ל-0.8 בתרחיש "עלייה", ול-0.29 בתרחיש "ירידה". יחס החוב ב-2036: 157% מול 154.5% בתרחיש הבסיס. הגרסה המתקדמת של הקירוב עשויה לצמצם את הפער, ובדיוק לכן הפיצ'ר מריץ אותה ומציג את התוצאה בגלוי, גם אם היא תראה שהפער קטן.

טכנולוגיות ותשתיות תעשייה משלימות

- **IMF MAC SRDSF (כמקור מתודולוגי):** מסגרת ניתוח קיימות החוב של IMF למדינות עם גישה לשווקים, שהתבנית שלה ציבורית. התרומה: הקירוב שלנו בנוי לפי המתודולוגיה המתועדת שלה, ולכן ההשוואה הוגנת ולא "איש קש".
- **Property-based testing עם Hypothesis:** ספריית בדיקות שמייצרת אלפי קלטים אקראיים ובודקת שתכונה מסוימת תמיד מתקיימת. התרומה: בדיקה שבתיק שבו כל האג"ח באותו אורך, שלוש מארבע השיטות חייבות להתכנס זו לזו. אם לא, יש באג.

מבנה נתונים

```
MethodRun { run_id; method: Literal["frozen", "jump", "imf_style", "imf_split", "bond"];
path: DebtPath }

MethodDelta {
  run_id: str
  method: str
  year: int
  interest_gap_pp: float      # method minus bond-level, % GDP
  debt_gap_pp: float
  gap_from_short_rollover: float
  gap_from_linkers: float
  gap_from_timing: float
}
```

קלט ופלט

קלט: snapshot ותרחיש. פלט: חמישה DebtPath, טבלת MethodDelta, והשנה עם הפער המקסימלי.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
הרצה מקבילה	concurrent.futures	בשרת ProcessPoolExecutor
בדיקות תכונה	Hypothesis	given(strategies)@

פיצ'ר 8: קיר הפקיעות האינטראקטיבי (Maturity Wall View)

מה הפיצ'ר עושה?

הממשק שבו המשתמש פוגש את כל זה. התצוגה המרכזית היא "קיר": כל אג"ח היא לבנה, הלבנים מסודרות לפי שנת פקיעה, וגובה כל עמודה הוא כמה חוב פוקע באותה שנה. מחוון התשואה צובע את הלבנים לפי כמה יקר יותר יעלה להחליף אותן. מתחת לקיר: מסלול יחס החוב ותשלומי הריבית, וההשוואה בין השיטות.

איך הפיצ'ר עובד?

צביעת הלבנים

לכל אג"ח מחושב "קפיצת הקופון": התשואה שבה היא תוחלף בתרחיש, פחות הקופון הנוכחי שלה. הצבע נקבע לפי סולם מתפצל (diverging) סביב אפס: גוונים קרים לאג"ח שיוחלפו בזול יותר, חמים לאג"ח שיוחלפו ביוקר. הסולם בטוח לעיוורי צבעים, וכל מידע שמועבר בצבע מופיע גם בטקסט (tooltip וטבלה).

אינטראקציה

- **מחוון תשואה:** שינוי מפעיל את מנוע המחזור מחדש (בדפדפן, ב-Web Worker כדי שהממשק לא יקפא), עם debounce של 150 מילישניות.
- **ריחוף או מיקוד על לבנה:** חלון צף עם המזהה, הקופון, תאריך הפקיעה, וההחלפה הצפויה.
- **מצב השוואה:** שתי מדינות זו לצד זו, על אותו ציר.
- **נגישות:** כל הלבנים נגישות במקלדת, Esc סוגר חלון צף, ולצד הקיר יש טבלה טקסטואלית מלאה של אותו מידע.

ההנחות צמודות לתוצאה

מענה לאיום "שימוש לרעה בתוצאה": כל גרף נושא כותרת משנה עם התרחיש וההנחות ("תשואה +1 נקודה, צמיחה 2%, גירעון ראשוני 2.9%"). צילום מסך של הגרף כולל אותן תמיד, לא כהערת שוליים. אין ציון ואין דירוג למדינה.

API ציבורי

לעיתונאים ולמפתחים: FastAPI עם endpoints כמו GET /v1/countries/{code}/snapshots/latest, POST /v1/runs (מקבל תרחיש ומחזיר run_id), GET /v1/runs/{run_id}. תיעוד אוטומטי ב-OpenAPI. מגבלת קצב לפי כתובת IP, ותוצאות שמורות במטמון לפי run_id, כי אותו קלט תמיד נותן אותה תוצאה.

טכנולוגיות ותשתיות תעשייה משלימות

- **D3.js:** ספריית הוויזואליזציה הנפוצה בעולם לגרפים מותאמים אישית, שבה משתמשים חדרי החדשות של The New York Times ושל The Economist עצמו. התרומה: קיר לבנים מותאם, שאי אפשר לבנות בספריית גרפים רגילה.
- **Web Workers:** מנגנון דפדפן להרצת קוד ברקע, בתהליכון נפרד מהממשק. התרומה: מנוע המחזור ב-Pyodide רץ בלי לתקוע את הגלילה והמחוון.
- **FastAPI עם slowapi:** מסגרת API מודרנית לפייתון שמייצרת תיעוד OpenAPI אוטומטית, ו-slowapi מוסיף לה הגבלת קצב. התרומה: מענה לאיום "הפלת השירות" בצד ה-API.

מבנה נתונים

```
WallBrick { instrument_id; maturity_year; face; coupon_pct; replacement_yield_pct; jump_pp }
WallColumn { year; total_face; bricks: list[WallBrick] }
ViewState { country; scenario_id; compare_country?: str; selected_brick?: str }
```

קלט ופלט

קלט: snapshot, Scenario, ו-DebtPath ו-MethodDelta מפיצ'רים 6 ו-7. פלט: תצוגה אינטראקטיבית, טבלה נגישה, ותשובות API.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
ויזואליזציה	D3.js	d3.scaleDiverging, d3.stack

שימוש	ספרייה	פעולה
postMessage / onmessage	Web Workers	ריצה ברקע
APIRouter, Limiter	FastAPI, slowapi	API

פיצ'ר 9: קישור משחזר וחותמת תוצאה (Reproducible Permalinks & Signed Results)

מה הפיצ'ר עושה?

לכל תוצאה יש קישור קבוע שמשחזר אותה במדויק, וחותמת דיגיטלית שמוכיחה שהיא יצאה מהמערכת. מענה לשני איומים ממודל האיום: "הכחשת מקור" (אי אפשר לדעת מאיפה הגיע מספר) ו"התחזות" (צילום מסך מזויף עם הלוגו של הכלי).

איך הפיצ'ר עובד?

מזהה הרצה דטרמיניסטי

run_id הוא SHA-256 של שלושה דברים יחד: snapshot_id, scenario_id, וגרסת המנוע. מכיוון ששלושתם דטרמיניסטיים, אותו run_id תמיד מתאר בדיוק את אותו חישוב. הקישור הוא פשוט /run/{run_id}. מי שפותח אותו מקבל את אותה תוצאה בדיוק, גם בעוד שנה, כי ה-snapshot לעולם לא משתנה (פיצ'ר 3).

חותמת תוצאה

כל תוצאה ארוזה כ-ResultBundle: ה-run_id, המספרים העיקריים, וההנחות. החבילה מסודרת ל-JSON קנוני ונחתמת בחתימת Ed25519 עם המפתח הפרטי של המערכת. המפתח הציבורי מפורסם באתר. כל אחד יכול לבדוק שמספר מסוים אכן יצא מהמערכת עם ההנחות האלה. תמונה שמורידים מהממשק נושאת בפינה קוד QR קטן לקישור, כך שצילום מסך מזויף אפשר לבדוק מול המקור.

ניהול המפתח

המפתח הפרטי לא יושב בקובץ על השרת: הוא נשמר בשירות ניהול מפתחות בענן, שמבצע את החתימה בתוכו ומחזיר רק את התוצאה. המפתח מתחלף פעם בשנה, וכל המפתחות הקודמים נשארים מפורסמים לבדיקת חתימות ישנות.

טכנולוגיות ותשתיות תעשייה משלימות

- **PyNaCl (Ed25519):** עטיפה לפייתון של libsodium. Ed25519 מהיר, החתימות קצרות (64 בייט), והוא הסטנדרט בכלים מודרניים. התרומה: חתימה ואימות במילישניות.
- **Cloud KMS (למשל AWS KMS או Google Cloud KMS):** שירות שבו המפתח הפרטי לעולם לא יוצא מהשירות. התרומה: גם פריצה מלאה לשרת לא מאפשרת לגנוב את המפתח.
- **Sigstore (כהשראה):** פרויקט פתוח לחתימה על תוכנה ותוצרים, עם יומן שקיפות ציבורי. אנחנו מאמצים את הרעיון של יומן: כל תוצאה חתומה נרשמת ביומן הוספה-בלבד (append-only), כך שגם המערכת עצמה לא יכולה "לחתום בדיעבד" על משהו ולהעלים אותו.

מבנה נתונים

```
ResultBundle {
  run_id: str
  snapshot_id: str
  scenario_id: str
  engine_version: str
```

```
  headline: { debt_2036_pct, interest_2031_pct, crossover_year }
  assumptions_text: str
}
SignedResult { bundle: ResultBundle; key_id: str; signature: bytes[64]; signed_at:
datetime }
```

קלט ופלט

קלט: תוצאות פיצ'רים 5 עד 7. פלט: קישור קבוע, חבילה חתומה, ורשומה ביומן השקיפות.

ספריות ופונקציות מרכזיות

שימוש	ספרייה / שירות	פעולה
SigningKey.sign(), ()VerifyKey.verify	PyNaCl	חתימה ואימות
Sign API, המפתח לא יוצא	Cloud KMS	שמירת מפתח
make(url)	qrcode	QR

פיצ'ר 10: רענון מתוזמן וזיהוי שינויים (Scheduled Refresh & Change Detection)

מה הפיצ'ר עושה?

מריץ את כל השרשרת (איסוף, נרמול, בדיקה, שמירה) אוטומטית אחרי כל פרסום של משרד חוב, ומתריע כשמשהו השתנה באופן חריג: קופצת חומת פקיעות, הנפקה גדולה חדשה, או טעינה שנכשלה בבדיקת השלמות.

איך הפיצ'ר עובד?

תזמון

כל מדינה מקבלת לוח זמנים לפי לוח הפרסום של המקור שלה (ה-MSPD האמריקאי וקובצי משרדי החוב באירופה מתפרסמים פעם בחודש, כל אחד בלוח משלו). הרענון רץ כ-workflow שבו כל שלב הוא משימה נפרדת, עם ניסיונות חוזרים ותלויות: אין שמירה בלי בדיקה, ואין בדיקה בלי הורדה.

מה נחשב שינוי חריג

אחרי כל טעינה מאושרת, הפיצ'ר משווה את המדדים העיקריים לטעינה הקודמת: גודל הקיר בשלוש השנים הקרובות, הקופון הממוצע של החוב שפוקע בשנתיים הקרובות, והריבית האפקטיבית. שינוי של יותר מ-3 סטיות תקן מהשינוי החודשי ההיסטורי יוצר התראה. טעינה שנדחתה (REJECTED) יוצרת התראה תמיד, כי היא אומרת שמקור השתנה ואולי המתאם צריך עדכון.

יומן שינויים ציבורי

מענה לאיום "שינוי ברירות מחדל": כל שינוי בהגדרות ברירת המחדל של התרחישים, בתמהיל ההנפקה או בגרסת המנוע נרשם ביומן שינויים ציבורי, עם תאריך, תיאור והסבר. ברירות המחדל עצמן נשמרות בקוד, כך שכל שינוי עובר code review.

טכנולוגיות ותשתיות תעשייה משלימות

- **Prefect:** מערכת תזמור workflows מודרנית לפייתון, שבה כל שלב הוא task עם retries, תלויות ותיעוד הרצות. התרומה: רענון שנכשל באמצע ממשיך מהשלב שנכשל, וכל הרצה מתועדת. חלופה פשוטה יותר: cron עם GitHub Actions.

- **Apprise:** ספרייה אחת ששולחת התראות לעשרות ערוצים (אימייל, Slack, Telegram). התרומה: ערוץ ההתראה הוא הגדרה ולא קוד.
- **Keep a Changelog:** מוסכמה מוכרת לתיעוד שינויים בפרויקטים פתוחים. התרומה: יומן שינויים שקורא אנושי מבין.

מבנה נתונים

```
RefreshRun { refresh_id; country; started_at; finished_at; status: Literal["OK",
"REJECTED", "FAILED"]; snapshot_id?: str }
ChangeAlert { alert_id; country; metric; old_value; new_value; z_score; created_at }
ChangelogEntry { entry_id; date; area: Literal["defaults", "mix", "engine", "adapter"];
description; commit_sha }
```

קלט ופלט

קלט: לוח זמנים לכל מדינה. פלט: snapshot חדש או התראה על כישלון, התראות על שינוי חריג, ורשומות ביומן השינויים.

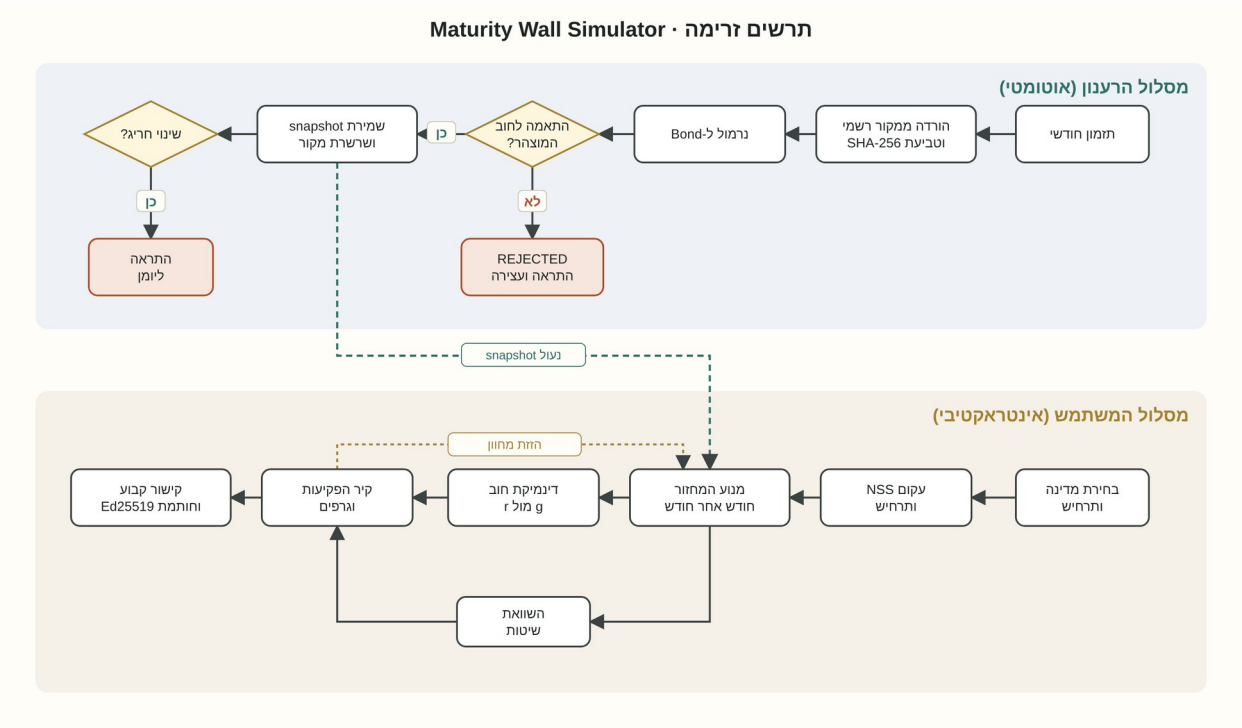
ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
@task(retries=3), flow	Prefect	תזמור
()Apprise().notify	Apprise	התראות
on: schedule: cron	GitHub Actions	תזמון פשוט

טבלת טכנולוגיות

פיצ'ר	טכנולוגיות ושפות תכנות	משאבים ושירותים חיצוניים
1. איסוף ממקורות	Python, httpx, tenacity, openpyxl, Pydantic	Fiscal Data API, ECB Data Portal, קובצי AFT ומשרד האוצר האיטלקי
2. נרמול ובדיקה	Python, decimal, Pandera, Polars	אין
3. מאגר ושרשרת מקור	DuckDB, Parquet, hashlib	אחסון קבצים (מקומי או S3)
4. עקום ותרחישים	NumPy, SciPy, QuantLib	פרמטרי ECB, תחזיות IMF והנציבות
5. מנוע המחזור	NumPy, Numba, Pyodide	אין (רץ גם בדפדפן)
6. דינמיקת חוב	SciPy, statsmodels	אין
7. השוואת שיטות	Python, Hypothesis	מתודולוגיית IMF MAC SRDSF
8. קיר הפקיעות	D3.js, Web Workers, FastAPI, slowapi	CDN לקבצים סטטיים
9. קישור וחותמת	PyNaCl, qrcode	Cloud KMS
10. רענון מתוזמן	Prefect, Apprise	שרת קטן או GitHub Actions

תרשים זרימה



התרשים מתאר את שני המסלולים של המערכת. מסלול הרענון (למעלה) רץ אוטומטית: הורדה, נרמול, בדיקת שלמות, ואז החלטה. טעינה שנכשלה מייצרת התראה ונעצרת, ואין לה דרך להגיע למאגר. טעינה שעברה נשמרת כ-snapshot, ומשם נבדקים שינויים חריגים. מסלול המשתמש (למטה) מתחיל בבחירת מדינה ותרחיש, עובר במנוע המחזור ובמכונת החוב, ובמקביל בהשוואת השיטות, ומסתיים בקיר, בקישור הקבוע ובחותמת. החץ החוזר מהמחוון אל מנוע המחזור הוא הלולאה האינטראקטיבית: כל הזזה מריצה את המנוע מחדש, בדפדפן.

מבנה בסיס הנתונים

נממש את מסד הנתונים באמצעות DuckDB, מסוג אנליטי עמודתי שרץ בתוך התהליך, וקבצים גולמיים באחסון לפי טביעת אצבע. כל הטבלאות הן הוספה-בלבד: אין UPDATE ואין DELETE.

טבלת fetches: כל קובץ גולמי שהורד

- **fetch_id:** מסוג VARCHAR, מפתח ראשי (UUID).
- **source_id:** מסוג VARCHAR, חובה (למשל "US_MSPD_T3").
- **url:** מסוג VARCHAR, חובה.
- **fetched_at:** מסוג TIMESTAMP, חובה.
- **raw_sha256:** מסוג VARCHAR(64), חובה. נתיב הקובץ נגזר ממנו.
- **http_status:** מסוג SMALLINT.

טבלת snapshots: כל טעינה מאושרת

- **snapshot_id:** מסוג VARCHAR(64), מפתח ראשי (SHA-256 של הרשימה הקנונית).
- **country:** מסוג VARCHAR(2), חובה.
- **as_of:** מסוג DATE, חובה.
- **created_at:** מסוג TIMESTAMP, חובה.

- **bond_count:** מסוג INTEGER, חובה.
- **total_face:** מסוג DECIMAL(20,2), חובה.
- **reconciliation_gap_pct:** מסוג DOUBLE, חובה, בין 0 ל-0.5.
- **adapter_version:** מסוג VARCHAR(40), חובה (git commit).
- **אילוץ:** ייחודיות על (country, as_of, snapshot_id).

טבלת snapshot_sources: קשר רבים-לרבים בין snapshot לקבצים

- **snapshot_id:** מפתח זר ל-snapshots.
- **fetch_id:** מפתח זר ל-fetches.
- מפתח ראשי משולב על שניהם.

טבלת bonds: האג"ח של כל snapshot

- **snapshot_id:** מפתח זר ל-snapshots, חובה.
- **instrument_id:** מסוג VARCHAR(12), חובה (CUSIP או ISIN).
- **kind:** מסוג ENUM('bill','fixed','linker','floater'), חובה.
- **coupon_pct:** מסוג DECIMAL(6,4), יכול להיות NULL ל-bill.
- **issue_date:** מסוג DATE, חובה.
- **maturity_date:** מסוג DATE, חובה, אילוץ CHECK שהוא אחרי issue_date.
- **face:** מסוג DECIMAL(20,2), חובה, אילוץ CHECK חיובי.
- **orig_tenor_m:** מסוג SMALLINT, חובה.
- **מפתח ראשי משולב:** (snapshot_id, instrument_id).

טבלת validation_issues: תוצאות הבדיקות

- **issue_id:** מסוג VARCHAR, מפתח ראשי.
- **refresh_id:** מפתח זר ל-refresh_runs.
- **rule_id:** מסוג VARCHAR, חובה.
- **severity:** מסוג ENUM('WARN','BLOCK'), חובה.
- **instrument_id:** מסוג VARCHAR, יכול להיות NULL (בדיקה על כל הטעינה).
- **detail:** מסוג VARCHAR, חובה.

טבלת curves: עקומי תשואות

- **curve_id:** מסוג VARCHAR(64), מפתח ראשי.
- **base:** מסוג ENUM('ECB_AAA','ECB_ALL','UST_PAR','FITTED'), חובה.
- **as_of:** מסוג DATE, חובה.
- **params:** מסוג STRUCT(beta0, beta1, beta2, beta3, tau1, tau2 DOUBLE), חובה.
- **spread_params:** אותו מבנה, יכול להיות NULL.
- **fit_rmse_bp:** מסוג DOUBLE.

טבלת scenarios: הגדרות תרחישים

- **scenario_id:** מסוג VARCHAR(64), מפתח ראשי (SHA-256 של ההגדרה הקנונית).
- **name:** מסוג VARCHAR, חובה.

- **curve_id:** מפתח זר ל-curves.
- **definition:** מסוג JSON, חובה (shocks, צמיחה, מאזן ראשוני, תמהיל).
- **is_default:** מסוג BOOLEAN, ברירת מחדל false.

טבלת runs: כל הרצה

- **run_id:** מסוג VARCHAR(64), מפתח ראשי (SHA-256 של snapshot, scenario וגרסה).
- **snapshot_id:** מפתח זר ל-snapshots.
- **scenario_id:** מפתח זר ל-scenarios.
- **engine_version:** מסוג VARCHAR, חובה.
- **created_at:** מסוג TIMESTAMP.

טבלת run_paths: המסלול השנתי של כל הרצה ושיטה

- **run_id:** מפתח זר ל-runs.
- **method:** מסוג ENUM('frozen','jump','imf_style','imf_split','bond'), חובה.
- **year:** מסוג SMALLINT, חובה.
- **debt_gdp_pct, interest_gdp_pct, effective_rate_pct, r_minus_g_pp:** מסוג DOUBLE.
- **מפתח ראשי משולב:** (run_id, method, year).

טבלת signed_results: יומן החתימות

- **run_id:** מפתח זר ל-runs, מפתח ראשי.
- **bundle:** מסוג JSON, חובה.
- **key_id:** מסוג VARCHAR, חובה.
- **signature:** מסוג BLOB (64 בייט), חובה.
- **signed_at:** מסוג TIMESTAMP, חובה.
- **prev_hash:** מסוג VARCHAR(64), חובה. טביעת האצבע של הרשומה הקודמת, כך שהיומן הוא שרשרת שאי אפשר להשמיט ממנה רשומה בלי שזה יתגלה.

טבלת refresh_runs, change_alerts ו-changelog

- **refresh_runs:** refresh_id (מפתח ראשי), country, started_at, finished_at, status (ENUM: OK, REJECTED, FAILED), snapshot_id (יכול להיות NULL).
- **change_alerts:** alert_id (מפתח ראשי), country, metric, old_value, new_value, z_score, created_at.
- **changelog:** entry_id (מפתח ראשי), date, area (ENUM: defaults, mix, engine, adapter), description, commit_sha.

מה האפיון סגר, ומה נשאר פתוח

- **נסגר:** שרשרת מקור מלאה מכל מספר לקובץ הרשמי (פיצ'רים 1 עד 3, 9), מענה לכל שישת האיומים ממודל האיום, והשאלה מה-POC הפכה לפיצ'ר שמודד את עצמו בכל הרצה (פיצ'ר 7).
- **נשאר פתוח:** הבדיקה שערכנו מראה פער ממשי מול קירוב בסגנון IMF בגרסה הפשוטה. האם הגרסה המתקדמת של הקירוב, שמפרידה חוב קצר וארוך, סוגרת את רוב הפער? אם כן, היתרון של המערכת מצטמצם לנגישות ולשקיפות בלבד.

- **נשאר פתוח:** המערכת יודעת בדיוק כמה כל מדינה תשלם ומתי. היא לא יודעת מי מחזיק את האג"ח האלה, ומי ייפגע כשהמחירים שלהן יזוזו.